Performance analysis of distributed SpMV using MPI and 1D modulo-cyclic row partitioning

Andrea Boarini
MAT. 244129
https://github.com/AndreaBoarini/PARCO-Computing-2026-244129
andrea.boarini@studenti.unitn.it

***Abstract*—Sparse Matrix–Vector multiplication (SpMV) is a fundamental kernel in scientific computing and machine learning, whose optimization is challenging due to irregular memory access patterns and low arithmetic intensity. This work investigates the scalability of an MPI-based distributed-memory SpMV implementation using a 1D modulo-cyclic row partitioning and a local CSR representation with explicit exchange of ghost vector entries. Experiments were conducted using up to 128 processes to evaluate strong and weak scaling, speedup, and parallel efficiency. Results show that, while the computational workload per process scales close to ideal, communication costs increase with the number of processes and become dominant at high concurrency, causing early saturation of strong-scaling speedup. Similarly, weak scaling exhibits non-ideal behavior, with growing communication time and volume per rank despite a constant local workload. Memory footprint analysis further indicates that an increasing fraction of per-process memory ($\sim 84\%$) is devoted to ghost data and communication buffers. Overall, these results demonstrate that the observed scalability limits are structural and originate from the 1D partitioning strategy, which balances computation but fails to preserve column locality. Alternative approaches, such as 2D partitioning schemes and topology-aware communication patterns, are therefore required to reduce communication overhead and improve scalability.**

I. Introduction

Problem. Sparse Matrix-Vector (SpMV) is a fundamental kernel block for many iterative solving algorithms and sparse linear algebra pipelines, widely adopted in scientific computation and different machine learning applications such as Deep Learning [1].

Despite its conceptual simplicity, SpMV introduces critical optimization challenges due to the irregular memory access pattern and low arithmetic intensity, factors which pose memory-bandwidth as the main application bottleneck. [2]
On distributed memory systems, this operation additionally requires explicit data partitioning and communication, forcing each participating process to retrieve the needed vector entries (ghost entries) so as to compute its own SpMV iteration on a CSR-based local matrix. [3]
Objectives. The main objectives of this work are to develop a distributed-memory SpMV algorithm using the OpenMPI library, implementing a 1D modulo-cyclic row partitioning as well as designing a communication pattern for remote entries exchanges. Finally, we evaluate both strong and weak scalability of the algorithm, further investigating its performances considering different metrics such as communication volume, workload balance and memory footprint.

Contributions. With this research we provide a reproducible protocol implementing a distributed CSR-based SpMV algorithm using MPI, together with a full spectrum analysis on the bottlenecks of such implementation and possible solutions to mitigate the non-negligible overhead.
All the algorithms and results can be viewed and reproduced at the official repository of this project. For reproducibility, please check `README.md`.

II. State of the Art

Performance of the SpMV is almost entirely characterized by a combination of the communication required to retrieve remote entries and the computation of the local sparse product. Each partitioning method to distribute the matrix impacts differently on the communication volume [3]. Common literacy choices fall between using 1D (block or cyclic) or 2D row partitioning strategies for distributing the matrix: 1D row-based ownership is often effective, while with 2D approaches trade higher implementation complexity for a reduction of the overall exchange volume [4].
In the context of this research we intentionally focus on the 1D modulo-cyclic partitioning in order to assess its limitations.

III. Research Methodology

A. Data structures

Each rank participating in the communication stores its own local representation of the matrix. The methodology depends on the specific partitioning algorithm chosen. For 1D modulo-cyclic distribution the *rows* of the matrix are distributed by the root rank using the following criterion [3]:

$$\text{owner}\ (g) = g \ \textbf{mod}\ P$$

With P being the total number of processes involved.
Each rank remaps its global row indices g to a packed local range by applying the following transformation to obtain the i-th row:

$$i = \left\lfloor \frac{g}{P} \right\rfloor$$

Each local representation is stored in a *Compressed Sparse Row* format (CSR), to ensure an efficient memory usage and speed up the multiplication kernel. [5]
We recall that the CSR matrix can be represented with 3 vectors Figure 1:

- `col_idx`: the indexes of the columns of the non-zero values.
- `values`: the non-zero values stored.
- `row_ptr`: the interval of the non-zero elements stored, for each row.

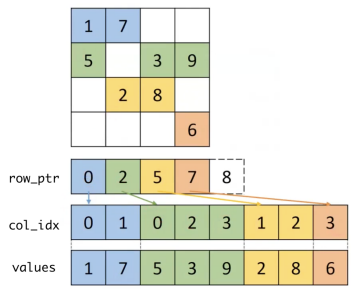

Fig. 1. CSR storing format example and its components.

The randomly generated input vector is initially distributed to the participating processes with the same logic used for the matrix. In this way each rank will end up having a single local structure `LocalX` with two separate vectors, one describing the possessed values (`owned`, Figure 2, gray areas in `vec`) and the other the needed indices that has to be exchanged and retrieved from other ranks (`ghost_idx`).
Once the ghost exchange and the local SpMV product are completed, the resulting vector (`local_y`) remains scattered among the processes and its dimension will depend on the local number of matrix rows (`N_local`) detained by each rank.

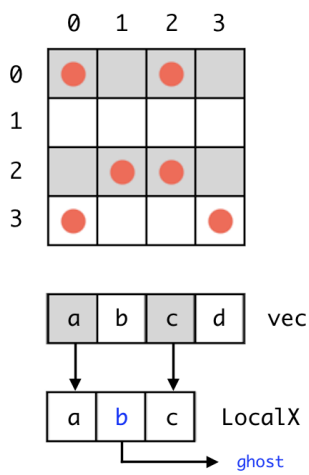

Fig. 2. Creation of the local vector from row-wise modulo-cyclic distribution and ghost entries identification.

B. Algorithm overview

Initial distribution. The root rank (rank 0), performs the initial distribution of both the matrix and the initial vector to the P MPI ranks. First it broadcasts the matrix metadata (dimensions and number of nz). In this way each rank can allocate the required local buffers. Then, rank 0 packs the nz into continuous send buffers by grouping each tuplet $(i, j, a_{i,j})$ according to the owner of its row (row-wise modulo-cyclic rule), computes counts and displacements and distribute the obtained chunks via the `MPI_Scatterv` collective [6]. The same logic is applied to the vector by packing the entries x_j instead. Each rank now holds its own local structure as a starting point.
Remote entries index discovery. To compute the SpMV, each rank needs only the subsets of vector entries whose global indices appears in the column list of its local CSR matrix (Figure 2, blue values, $d =$ vec[3] is not considered as ghost). Therefore, the needed entries are identified by scanning the local CSR structure and marking the global column indices referenced by at least one local nz. The marked indices that aren't already owned by the current ranked are then inserted into the ghost list `ghost_idx` (1).

Algorithm 1 ghost entries discovery (rank-wise)

```
is_needed [0 ... N-1] = false
for i ← 0 to N_local − 1 do
    for j ← row_ptr[i] to rowPtr[i + 1] − 1 do
        g = col_idx[j]
        is_needed[g] = true
    end for
end for

// ghost_idx is empty
for g ← 0 to N − 1 do
    if is_needed[g] and (g mod P ≠ rank) then
        append g to ghost_idx
    end if
end for
n_ghost = |ghost_idx|
```

Ghost exchange - halo communication. In this section each rank performs an all-to-all communication to fetch the values corresponding to the remote indices previously obtained. Initially, ghost indices are grouped by their own rank using the cyclic rule; processes then exchange request counts and requested global indices exploiting `MPI_Alltoall` and `MPI_Alltoallv` MPI collectives [6]. Note that each rank answers to requests from others by packing the requested values from its local owned segment applying each time the transformation from local to global indexing: $g \rightarrow \lfloor g/P \rfloor$. A second `MPI_Alltoallv` returns the actual values to the interested ranks. The obtained values are reordered so as to match the original `ghost_idx` layout.
Index remapping and SpMV kernel. After all the needed values are correctly retrieved, the possessed values (`owned` and `ghos_entries`) are merged into a single structure `merged_local_X` within each rank. After that, a *global-to-local* mapping that assigns consecutive local positions to the owned global indices in `ghost_idx` is created. Using this map we remap the CSR local indices from global to local offsets in `merged_local_X`. As a result, in the local SpMV

section (2), we avoid runtime searches induced by global indexing, mitigating the cost of the computation.

Algorithm 2 remapped SpMV kernel

```
for i ← 0 to N_local − 1 do
    s = 0
    for j ← row_ptr[i] to row_ptr[i + 1] − 1 do
        s += val[j] × merged[col_idx_loc[j]]
    end for
    local_y[i] = s
end for
```

As mentioned, `column_idx_local` indexes directly into `merged_local_X` thanks to the previous mapping step.

IV. Experimental setup

1) Environment architecture: The research presented has been entirely conducted on the University of Trento's HPC cluster, by allocating 4 different nodes, each of them having a total of 96 occupable physical cores. The accurate specifications of a single node are:

- **CPU**: Intel(R) Xeon(R) Gold 6252N CPU @ 2.30 GHz (x86_64)
- **Layout**: 4 NUMA nodes with 24 cores each
- **OS**: Rocky Linux 9
- **Cache hierarchy**: L1-32KB, L2-1MB, L3-36.6MB (per core)

2) Job allocation: To guarantee full reproducibility of the measurements and reduce experimental noise, process placement was controlled explicitly. This also avoids performance variability due to suboptimal operating system scheduling. For this reason processes were distributed following the `place=scatter:excl` policy with a number of working units per node which depends on the MPI processes requested for a specific run.

Ranks were mapped according to the NUMA topology of each node using `-map-by numa` and bound to individual CPU cores using `-bind-to core`, enforcing a one-rank-per-core policy. Explicit binding and NUMA-aware mapping are particularly important for SpMV kernels, which are memory-bandwidth-bound and sensitive to non-uniform memory access effects [7]. Overall, this placement strategy ensures consistent core affinity, balanced load across nodes, and stable performance measurements across scaling experiments.

The `mpich 3.2.1` compiler wrapper based on `GCC 9.1.0` was used with `-O3` optimization.

3) Dataset: In the scope of this research both real sparse matrices and synthetic generated matrices (with random sparsity pattern) were used: the former for testing strong scaling while the latter for evaluating weak scaling.

Concerning real matrices, samples with different degree of sparsity, dimension and pattern structure were considered. I For weak testing, instead, the generated matrices ranged from 2^8 to 2^{15} rows, all of them sharing the same density structure with 32 *nz* per row. In this way the workload for each rank is kept constant, while the number of processes increases.

TABLE I
Real matrices intrinsics

Name	N = M	nnz	dimension	symmetry*
torso3	259K	4430K	medium	95.1%
bcsstk15	3,98K	117K	small	100.0% **
vas_stokes_4M	4380K	132000K	large	46.4%
af_shell7	505K	17580K	medium	100.0% **

*Rrefers to *pattern symmetry*
**For matrices with 100% symmetry, due to *mmio* library limitations, we analyze only one half of the structure, approximately halving the real number of *nz* considered

4) Metrics used: The algorithm evaluation considers both computational efficiency and communication overhead. Computation and communication times are measured separately to isolate their contributions to the total runtime. Scalability is further assessed through speedup and parallel efficiency with respect to the single-process execution. Communication behavior is analyzed using per-rank data exchange volumes, while load balance is evaluated through the distribution of nonzeros across processes. Memory usage per rank is also monitored to assess the scalability of the data structures and the overall resource footprint. These metrics are retrieved as follows:

- **Speedup**: $S(p) = T(1)/T(p)$
- **Efficiency**: $E(p) = S(p)/p$
- **Memory footprint**: M_p (in KB), sum of the size of all structures and *nz* allocated by rank p.
- **Exchange volume**: V_p, total number of remote entries sent and received by process p.
- **Load**: nnz_p, total number of nz values owned by rank p.

V. Results

Strong scaling analysis returned significant results.

At low process counts, speedup increases as the local

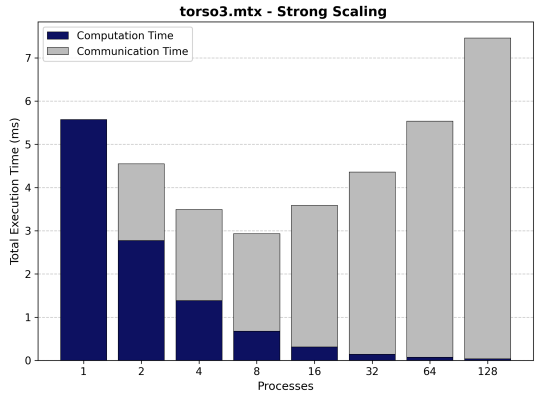

Fig. 3. Strong scaling time execution breakdown for *torso3*

computation workload decreases, behavior observed in the computation vs. communication times breakdown Figure 3: we illustrate the case of matrix *torso3*; similar trends are

observed for the others. By keeping the problem's size fixed and increasing the processes, the computation time exhibits near-ideal scaling, confirming that the local SpMV workload is evenly distributed across ranks. Oppositely, when more ranks are involved in the communication, speedup rapidly saturates and efficiency drops due to the fact that the communication component fails to scale accordingly, with the direct consequence of dominating the total runtime Figure 4. The observed pattern is intrinsically associated with 1D row-modulo cyclic partitioning. More specifically, 1D partitioning successfully balances the *nz* elements among different ranks, but still fails to fully preserve column locality, causing each rank to depend on an increasing number of remote entries as *P* grows [3] [8]. In Figure 4, we notice that this aspect extremely impacts on matrix *bcsstk15*'s speedup, being further aggravated by:

- Considering only half of the symmetric structure (Table I $**$), column locality deteriorates.
- The increased communication-to-computation ratio, due to sample matrix's small size.

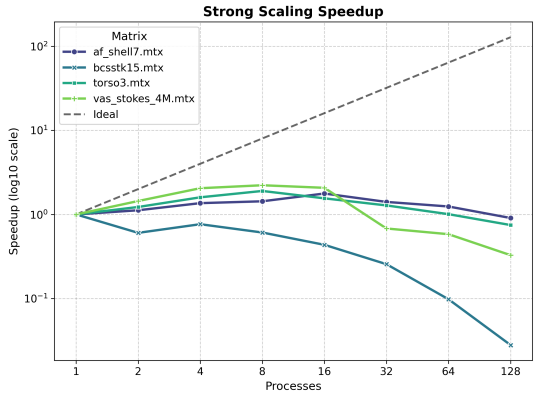

Fig. 4. Speedup trend for strong scaling of real samples

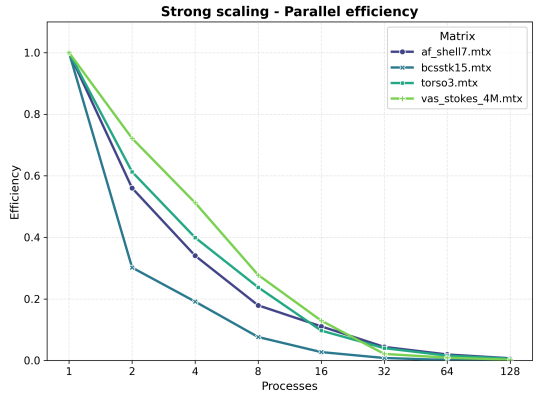

Fig. 5. Parallel efficiency trend for strong scaling of real samples

To further analyze 1D rows partitioning inefficiency, we tested the variability of the normalized communication volume per rank, defined as the ratio between the *average communication volume* and the *average computational load* among *P* increasing processes, in a weak scaling setup.

$$C_{norm} = \frac{\frac{1}{P}\sum_{p=1}^{P} V_p}{\frac{1}{P}\sum_{p=1}^{P} nnz_p} \tag{1}$$

For this setup, by construction, the average *nnz* per rank is

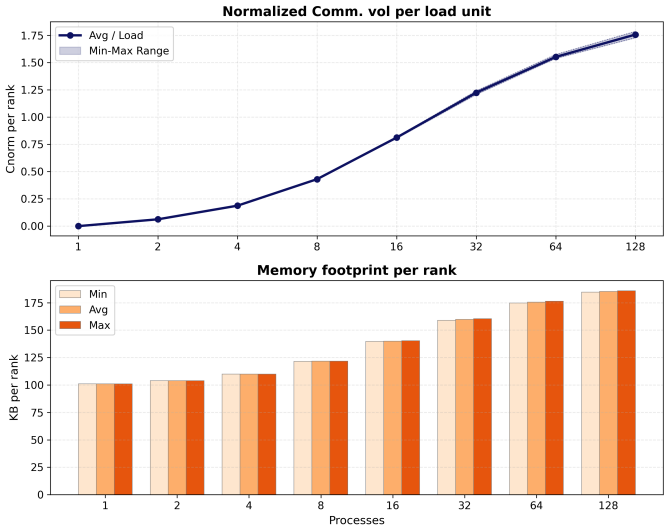

Fig. 6. Normalized communication volume per rank (above), Memory footprint in KB per rank (below)

kept constant and, under these conditions, a well-designed data partitioning algorithm is expected to limit the average communication volume's growth as *P* increases [3] [9]. This isn't observed when 1D partitioning is adopted, suggesting, as predicted, that matrix rows owned by a process increasingly reference columns owned by other processes, leading to a growing set of ghost values which eventually leads the kernel to a communication-bound regime (Figure 6, above).
Partitioning strategy's effect on exchange volume also affects negatively memory allocation [9]. In weak scaling environment, the memory allocated when $P = 1$ can be considered as a solid baseline to evaluate the memory overhead induced by an increment of exchanges when $P > 1$, since, for the former, only local structures are accounted. Results show that the memory overhead paid due to the growth of communication reaches $\sim 84\%$ per rank (Figure 6, below), entirely associated with the allocation of ghost entries and communication related buffers. Eventually, this confirms that the scalability limitations of 1D modulo-cyclic row partitioning affect not only runtime but also memory efficiency.

VI. Conclusions and Future Work

This work investigated the scalability of a distributed-memory SpMV implementation based on a 1D modulo-cyclic row partitioning. The results show that, although this strategy provides good computational load balance, it induces a sub-optimal communication pattern, leading to early saturation of strong scaling speedup and reduced parallel efficiency. Weak

scaling experiments further revealed an increasing normalized communication volume per rank and a growing memory overhead dominated by ghost entries, indicating that the observed limitations are structural and originate from the partitioning strategy itself.

Future work will focus on a 2D partitioning approach, which distributes both rows and columns across a $P_r \times P_c$ process grid. This algorithm is known to reduce the number of communication partners per rank and limit communication growth to $O(\sqrt{P})$ (vs. $O(P)$ for 1D) improving scalability for large process counts [3] [10].

Additional directions would also include structure-aware matrix reordering to improve locality [11] and the use of neighborhood collectives (e.g. `MPI_neighbor_alltoallv`) to reduce communication partners [12].

REFERENCES

[1] Y. Saad, Iterative Methods for Sparse Linear Systems, 2nd ed. Philadelphia, PA, USA: SIAM, 2003.

[2] S. Williams, L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel, "Optimization of sparse matrix-vector multiplication on emerging multicore platforms," Proceedings of the IEEE, vol. 97, no. 2, pp. 261–293, Feb. 2009.

[3] A. Buluç and J. R. Gilbert, "Parallel sparse matrix-vector multiplication on distributed-memory systems," SIAM Journal on Scientific Computing, vol. 34, no. 4, pp. C170–C191, 2012.

[4] D. Irony, S. Toledo, and A. Tiskin, "Communication lower bounds for distributed-memory matrix multiplication," Journal of Parallel and Distributed Computing, vol. 64, no. 9, pp. 1017–1026, Sep. 2004.

[5] I. S. Duff, A. M. Erisman, and J. K. Reid, Direct Methods for Sparse Matrices, 2nd ed. Oxford, U.K.: Oxford Univ. Press, 2017.

[6] https://www.open-mpi.org

[7] G. Hager and G. Wellein, Introduction to High Performance Computing for Scientists and Engineers. Boca Raton, FL, USA: CRC Press, 2010.

[8] E. Ballard, A. Buluç, J. Demmel, L. Grigori, B. Lipshitz, O. Schwartz, and S. Toledo, "Communication optimal parallel multiplication of sparse random matrices," SIAM Journal on Scientific Computing, vol. 38, no. 4, pp. A1643–A1666, 2016.

[9] E. Ballard, J. Demmel, O. Holtz, and O. Schwartz, "Minimizing communication in numerical linear algebra," SIAM Journal on Matrix Analysis and Applications, vol. 32, no. 3, pp. 866–901, 2011.

[10] J. Dongarra, I. S. Duff, D. C. Sorensen, and H. A. van der Vorst, Numerical Linear Algebra for High-Performance Computers. Philadelphia, PA, USA: SIAM, 1998.

[11] A. George and J. W. H. Liu, Computer Solution of Large Sparse Positive Definite Systems. Englewood Cliffs, NJ, USA: Prentice Hall, 1981.

[12] T. Hoefler, A. Lumsdaine, and W. Rehm, "Implementation and performance analysis of non-blocking collective operations for MPI," in Proc. IEEE/ACM SC, 2007.